

Composition Pair: Three Adjacencies × Hybrid Systems Composition — Adjacency-Pattern Mapping as the Architectural Property That Specifies Which Adjacency Operates Through Which Composition Pattern in CKS

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 6 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the emergent architectural property — *adjacency-pattern mapping* — that arises when the three-adjacencies commitment composes with the hybrid-systems-composition commitment, so that downstream work can adopt, vary, or argue against the resulting specification without leaving the per-instance pattern position of any adjacent AI component undocumented.

Abstract

The CKS pattern carries two foundational commitments that, taken individually, do not specify how the objects they each name relate to one another. The three-adjacencies commitment names three architectural neighbors with which CKS is most often conflated — retrieval-augmented generation (RAG), parametric (in-weight) memory, and external structured memory of the Knowledge Objects (KO) and OIDA family — and fixes the boundary between CKS and each. The hybrid-systems-composition commitment names three legitimate composition patterns — Pattern A (adjacent component as input to a cell), Pattern B (adjacent component as derived view of substrate content), and Pattern C (adjacent component as separate concern) — and specifies that each adjacent AI component must occupy one of these positions. Neither commitment alone specifies which patterns are admissible for which adjacencies. Their composition produces an emergent architectural property — *adjacency-pattern mapping* — in which each adjacency type carries a documented relationship to each pattern, such that canonical, admissible, and inapplicable positions are architecturally specified rather than left to deployment discretion. This note formalizes adjacency-pattern mapping as a standalone composition-pair object: its four operational components, what the composition forces beyond either commitment alone, the anti-patterns that violate it, and the operational test by which a deployment's mapping coherence is verified.

1. Why the composition pair needs to be formalized as standalone

Each of the two foundational commitments is well-defended on its own. The three-adjacencies commitment defends a position about *what CKS is not* by fixing the architectural boundary against three adjacent objects. The hybrid-systems-composition commitment defends a position about *how CKS coexists with adjacent objects* by naming three patterns and ruling out a fourth. In isolation, each commitment is operationally complete for the question it answers. In composition, however, a question arises that neither alone answers: when an adjacent AI component is positioned in a CKS deployment, *which patterns are admissible for which adjacency types, and on what architectural grounds?*

The question is consequential. A real CKS deployment commonly carries instances of all three adjacencies simultaneously — a RAG index over reference documents, a fine-tuned LLM specialized to a domain vocabulary, and an external structured store inherited from a prior knowledge initiative. The hybrid-systems-composition commitment alone tells the deployment that each component must occupy Pattern A, B, or C; it does not say which the structured-memory instance should occupy, nor whether the RAG index can sensibly occupy Pattern B. The three-adjacencies commitment alone tells the deployment what each adjacency *is*; it does not say which compositional positions each can occupy. Without an explicit mapping, a deployment satisfies both commitments in form while leaving per-instance position decisions ad hoc — the configuration in which composition anti-patterns most reliably emerge.

A standalone derivation is warranted because the composition produces an architectural specification consequential for the same patentable territory as A1.14 and A1.16 individually — "AI architecture," "AI orchestration patterns," "hybrid AI systems with governance" — but not reducible to either parent. A note deriving only from A1.14 leaves the composition gap open; a note deriving only from A1.16 leaves the adjacency-specific applicability open. The composition-pair note closes both at once. A1.14 × A1.16 also opens the third tier of Phase A4: subsequent A1.14 cluster notes (adjacency-vs-substrate distinction, AI mediation across adjacencies, adjacency composition requirements) refine, distinguish, or close around the mapping derived here.

2. The emergent architectural property: adjacency-pattern mapping as four operational components

When A1.14 and A1.16 compose, the resulting specification has four operational components.

Each adjacency carries a documented relationship to each composition pattern. For each of the three adjacency types, and for each of the three patterns, the relationship is one of: *canonical* (the position the adjacency was structurally designed for, and the default a deployment should adopt absent specific reasons otherwise), *admissible* (a position the adjacency can occupy with explicit architectural justification), or *inapplicable* (a position the adjacency cannot occupy because its structural shape is incompatible with the pattern's requirements).

The mapping the composition supports is asymmetric across adjacency types. RAG canonically occupies Pattern A — a cell consults a RAG index during execution and writes substrate state under its rule, with the consultation recorded as substrate provenance — and admissibly occupies Pattern C as reference-document retrieval entirely outside coordination scope. RAG is inapplicable to Pattern B: the pattern requires the adjacent component to be derivable from substrate state, and a RAG index over an

external corpus is not so derived; a vector index built over substrate content for semantic search is a different object that occupies Pattern B in its own right. Parametric memory canonically occupies Pattern A — a cell uses a fine-tuned mediator whose parametric content improves domain fluency, with the substrate remaining authoritative — and admissibly occupies Pattern C for tasks the substrate does not represent. Parametric memory is inapplicable to Pattern B because parametric content cannot be regenerated from substrate state without retraining, which is not the dynamic-derivation property Pattern B requires. External structured memory of the KO/OIDA family canonically occupies Pattern A or Pattern B depending on its origin relative to the substrate: when an inherited structured store is consulted by a cell as typed input, Pattern A is canonical; when substrate content is projected into typed structured form to support large-scale typed queries, Pattern B is canonical. Structured memory admissibly occupies Pattern C outside coordination scope. The asymmetry across adjacency types is the architectural content of the mapping.

The relationship specifies the canonical pattern for each adjacency. Among admissible positions, one is canonical and is the default a deployment should adopt; departures must be justified architecturally. The canonical-vs-admissible distinction is what allows architectural review to surface decisions that need justification: a RAG index positioned as Pattern C is not wrong, but the deployment should be able to say why source-document grounding is being held outside coordination scope.

The relationship determines what cross-adjacency operations are legitimate. Operations involving more than one adjacency are not architecturally neutral. A cell consulting both a RAG index (Pattern A) and a fine-tuned mediator (Pattern A) operates two consultations of the same architectural position; provenance must record both, and the cell's rule must authorize both. A cell whose Pattern A consultation of a structured store informs a write that triggers a Pattern B view rebuild exercises two distinct positions in sequence; the boundary between consultation and view-update must remain inspectable. The mapping makes these operations legitimate by specifying the position each component occupies and the requirements each position imposes.

A2.95 composition anti-patterns become identifiable through adjacency-pattern mismatches. The three anti-patterns the source paper supports — adjacent component as substrate substitute (A3.23), adjacent component as ungoverned writer (A3.21), and hidden bidirectional coupling (A3.22) — each produce a recognizable mismatch. The substitute anti-pattern emerges when a Pattern B derived view drifts into the position the substrate canonically holds. The ungoverned-writer anti-pattern emerges when an adjacency in Pattern A is granted write authority that exceeds the cell's rule. The hidden-coupling anti-pattern emerges when a component is simultaneously consulted (Pattern A) and updated by (Pattern B) substrate without an orchestration rule mediating either direction. The mapping makes each mismatch nameable at the moment a practical decision is being made.

3. What the composition forces beyond either commitment alone

The composition forces four properties neither parent provides. *Adjacency-pattern correspondence as architectural commitment* — every adjacency instance is positioned in a pattern admissible for its type, with inapplicable combinations (RAG as Pattern B, parametric memory as Pattern B) ruled out at the architectural level rather than as a runtime check. *Cross-adjacency operations under explicit specification* — when more than one adjacency is consulted in a single cell execution, or when one adjacency's output flows into another through substrate state, the orchestration rule records which

adjacencies were consulted, in what positions, and with what role in the reasoning that produced the writes. *Deployment architecture decisions guided by the mapping* — a new RAG index added to an existing deployment is positioned canonically as Pattern A unless deployment-specific reasons argue otherwise, and the decision is made in architectural review rather than left to whoever wires the integration. *Vendor classifications mapped to CKS architectural lens* — when a vendor characterizes its product as "knowledge graph," "memory layer," or "retrieval system," the deployment is responsible for naming, in CKS terms, which adjacency type the product actually is and which pattern position it occupies, which is what allows the architecture to remain coherent across vendor changes.

4. Anti-patterns that violate the composition

Four anti-patterns specifically violate adjacency-pattern mapping. They are operationally distinct from the A2.95 anti-patterns the hybrid-systems-composition commitment defines: those three are about an adjacent component acquiring an unnamed position; these four are about the *mapping itself* failing.

Adjacency-pattern mismatch. A deployment positions an adjacency in a pattern inapplicable for its type — most commonly a RAG index treated as Pattern B when the index is over an external corpus the substrate does not derive, or a fine-tuned LLM treated as Pattern B when the model's parametric content cannot be regenerated without retraining. The mismatch is invisible at runtime — the deployment functions — but the architectural commitment Pattern B preserves (the view is regeneratable from substrate state) is silently broken.

All-adjacencies-same-pattern. A deployment forces every adjacency through one pattern, typically Pattern A, on the grounds that "all our AI components are inputs to cells." The framing collapses the architectural distinctions among adjacencies and erases cases in which Pattern B (substrate-derived view) or Pattern C (separate concern) is the architecturally correct position. The deployment may satisfy A1.16's surface requirement that every component occupy *some* pattern while failing to make pattern-position decisions that respect each adjacency's architectural shape.

Vendor-determined adjacency classification. A deployment accepts a vendor's classification of its own product without independently mapping it to CKS adjacency categories. Vendor categories are designed for the vendor's framework, not for CKS: a "memory layer" in a vendor's vocabulary may architecturally be parametric memory, external structured memory, or something the deployment is treating as substrate while the architecture does not. The mapping fails because the adjacency type is misnamed before the pattern position is even selected.

A2.95 anti-patterns compounded with mismatches. When an A2.95 anti-pattern is present alongside an adjacency-pattern mismatch, the failures reinforce each other. A RAG index treated as Pattern B (mismatch) that becomes the surface cells reach for first (substitute drift) is not a Pattern B violation that grew into a substitute violation; it is a single configuration in which the architecture lost coherence at two layers simultaneously, and unwinding either failure without the other does not restore the architecture.

5. What adjacency-pattern mapping is NOT

Not A1.14 alone. The three-adjacencies commitment fixes the boundary against RAG, parametric memory, and external structured memory. It does not specify how any of the three is positioned when present. The mapping presupposes the boundary; it adds the per-instance pattern position the boundary does not provide.

Not A1.16 alone. The hybrid-systems-composition commitment fixes the three legitimate positions any adjacent component can occupy. It does not specify which positions are canonical, admissible, or inapplicable for which adjacency type. The mapping presupposes the three positions; it adds the adjacency-specific applicability.

Not implicit adjacency-pattern correspondence. A deployment in which the per-instance position of each adjacency is "obvious to those who built it" but undocumented does not satisfy the mapping. The architecture's commitment is to inspectable positions, not to positions stable in the heads of current operators. Implicit correspondence fails the inspect right (§3.3) at the moment the team turns over.

Not vendor-determined mapping. The mapping is a CKS architectural property and must be expressed in CKS terms. Vendor frameworks may inform the mapping but cannot determine it.

6. Operational test

A hybrid CKS deployment satisfies adjacency-pattern mapping if and only if all of the following are true at all times during the deployment's existence.

(e.1) Adjacency-pattern documentation. For every adjacent AI component, the deployment's architectural documentation records (i) the CKS adjacency type the component instantiates — RAG, parametric memory, or external structured memory — and (ii) the pattern position the component occupies — Pattern A, B, or C. The documentation is inspectable by any human exercising the inspect right (§3.3) without scheduling, approval, or runtime intermediation. A component lacking either field, or whose field is filled by a vendor category rather than a CKS architectural category, fails the test.

(e.2) Cross-adjacency-operation specification. Every cell whose execution involves more than one adjacency, or whose writes affect a component held in another pattern position, is governed by an orchestration rule that explicitly authorizes the cross-adjacency operation. Provenance recorded for the cell's writes identifies each adjacency consulted, the position it occupies, and the role the consultation played in the reasoning that produced the writes. A cross-adjacency operation not under explicit rule authorization, or whose writes do not record per-adjacency provenance, fails the test.

(e.3) Mapping coherence with A2.95. No component occupies a pattern position inapplicable for its adjacency type, and no combination of pattern positions reproduces an A2.95 anti-pattern. The two checks are run together rather than separately because the failure modes interact: a RAG-as-Pattern-B mismatch combined with a substitute drift produces a configuration neither failure produces alone. Any inapplicable position, or any A2.95 anti-pattern present alongside the mapping, fails the test.

A deployment that satisfies (e.1)–(e.3) instantiates adjacency-pattern mapping in the sense the composition produces. A deployment that fails any of the three may satisfy A1.14 and A1.16 individually while failing their composition.

One-sentence test. If every adjacent AI component carries an architecturally documented adjacency-type-plus-pattern-position pair, and every cross-adjacency operation is governed by an orchestration rule whose provenance records the per-adjacency consultations, the deployment instantiates adjacency-pattern mapping; if either condition fails, it does not.

7. Why naming the composition as standalone matters

Adjacency-pattern mapping is the architectural property that makes hybrid CKS deployments auditable beyond the per-component-position level the hybrid-systems-composition commitment supplies. Without the mapping, a deployment can satisfy A1.14 by acknowledging that its components are RAG, parametric memory, and structured memory, and can satisfy A1.16 by stating that each component is in some pattern — and still leave the architectural correspondence between adjacency type and pattern position undefended. The composition closes that gap by making the correspondence itself a property the architecture commits to.

Naming the composition as standalone matters for three further reasons. It opens the A1.14 cluster of Phase A4: subsequent A1.14 × A1.08 (adjacency-vs-substrate distinction) presupposes the mapping when distinguishing adjacent components from the substrate beside them; A1.14 × A1.04 (AI mediation across adjacencies) presupposes the mapping when specifying the mediator role at each pattern position; A1.14 × A1.13 (adjacency composition requirements) closes the cluster by tying the mapping into the composition-requirements framework. It supports the human-selective composition coherence formalized earlier in Phase A4 by giving the human a specification to select against — *which* adjacencies in *which* positions — rather than a list of options without architectural correspondence. And it distinguishes CKS from systems with implicit adjacency classification, in which the architectural type of any component is whatever the integration team or vendor catalog says it is, with no architectural test for whether the classification is right.

Subsequent work that adopts the CKS pattern, composes it with adjacent AI components, or argues against it should treat adjacency-pattern mapping as the property that links the three-adjacencies and hybrid-systems-composition commitments into a single architectural specification. Subsequent work that operates on the parents in isolation is using a weaker specification, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Related notes in this series

Li, W. (2026). *Three Adjacencies: Why CKS Is Not RAG, Not Parametric Memory, and Not External Structured Memory*. 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Three Positions, Not Four: Composing CKS Substrates with RAG Indexes, Fine-Tuned Models, Vector Databases, and External Knowledge Stores*. 30 April 2026. ORCID:

0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Composition Pair: Three Adjacencies × Hybrid Systems Composition — Adjacency-Pattern Mapping as the Architectural Property That Specifies Which Adjacency Operates Through Which Composition Pattern in CKS*. 6 May 2026. ORCID: 0009-0004-8065-3235.